# Neural Networks and Deep Learning Lab-5

Pakalapati Naga Srinivasa Varma

February 16, 2026

## 1 Objective

The objective of this lab assignment is to implement and train a Convolutional Neural Network (CNN) for handwritten digit classification using the MNIST dataset. The experiment aims to analyze the effect of architectural choices, dropout regularization, and hyperparameter tuning on model performance.

## 2 Dataset Description

The MNIST dataset consists of 60,000 training images and 10,000 test images. The training set was further split into 54,000 training samples and 6,000 validation samples.

**Preprocessing steps:**

- Pixel values were normalized to the range [0,1] using `ToTensor()`.
- The original training set was split into training and validation sets in a 90:10 ratio.
- Data was loaded using PyTorch DataLoaders with batch size 64.

## 3 CNN Architecture

The initial CNN model consisted of:

- Two convolutional layers (3×3 kernels)
- ReLU activation after each convolution
- Max pooling layers for spatial downsampling
- Output layer with 10 neurons for multi-class classification

The model was trained using Cross-Entropy Loss and the Adam optimizer.

## 4 Training and Validation Performance

Figure 1 shows the training and validation loss and accuracy curves over epochs.

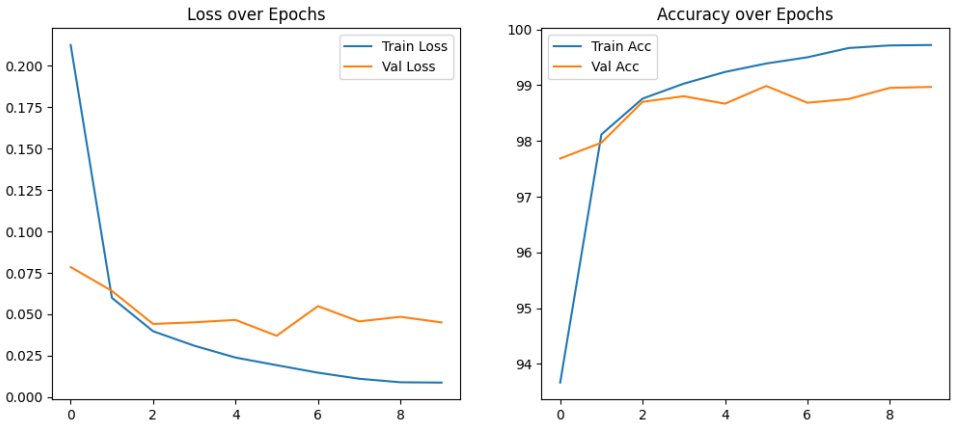


Figure 1: Training and Validation Loss and Accuracy

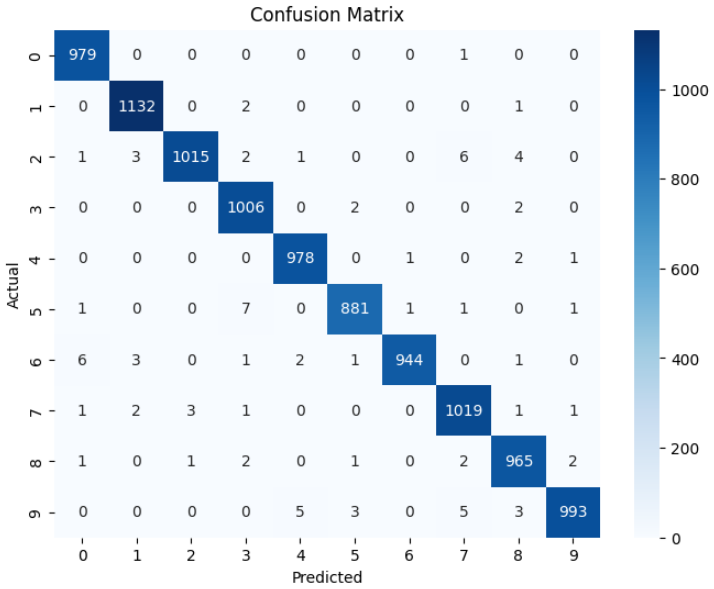


Figure 2: Confusion Matrix on Test Set

It can be observed that the training accuracy continues to improve while the validation accuracy saturates, indicating the onset of mild overfitting.

The Test Accuracy observed is 99.12%.

## 5 Misclassification Analysis

Some misclassified samples are shown in Figure 3. These errors mainly occur for visually similar digits.

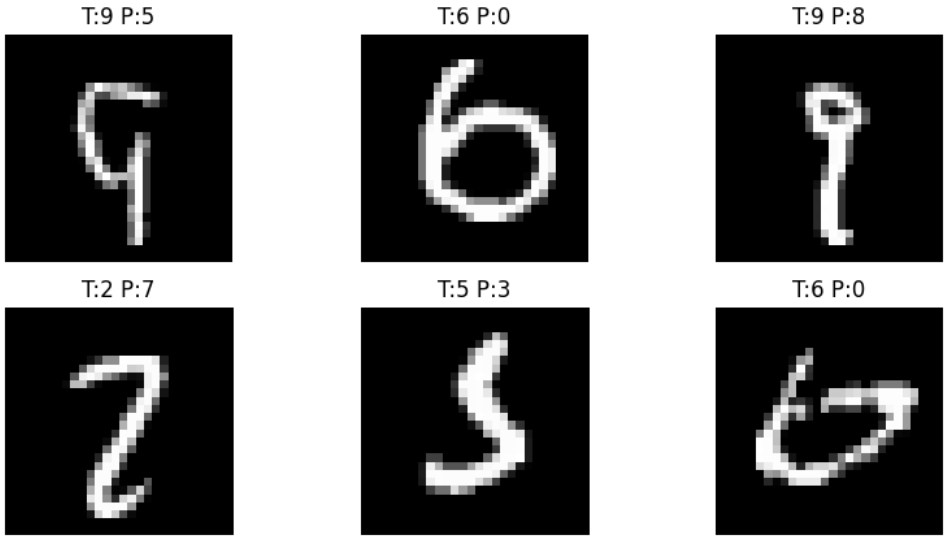


Figure 3: Sample Misclassified Images

## 6 Architecture Comparison

Different CNN configurations were evaluated by varying:

- Number of convolutional layers
- Number of filters
- Activation functions (ReLU vs Leaky ReLU)

### 6.1 Convolution Layers and Activation function effect:

First, observe the effect of number of convolution layers and activation functions on the model:

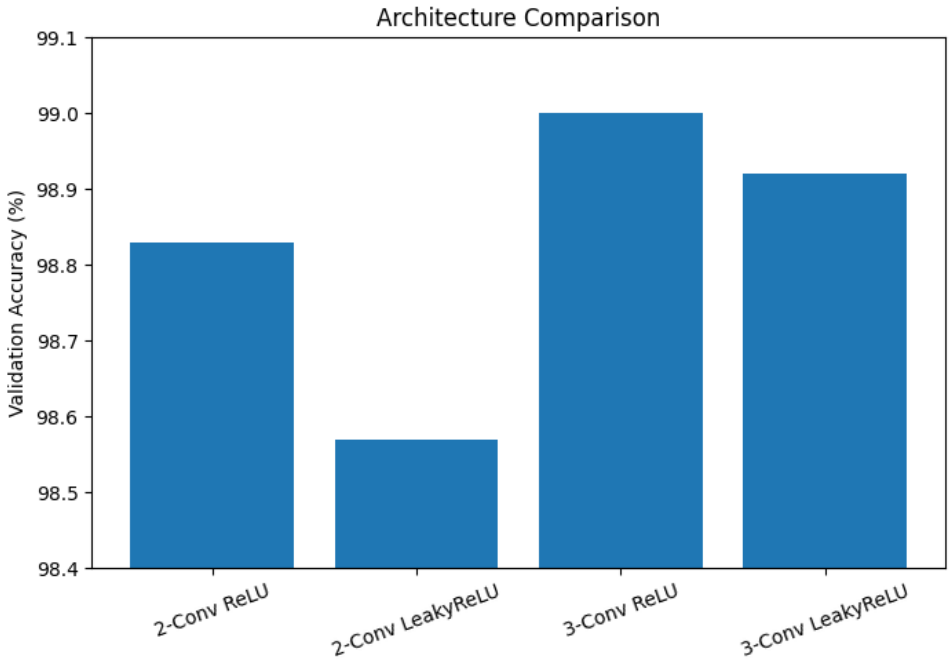


Figure 4: Effect of Convolution Layers and Activation Function

We observe that the validation accuracy slightly improves as the number of convolutional layers increases. The performance of LeakyReLU is marginally lower than ReLU, although the difference is small.

### 6.2 Overfitting Analysis Using Dropout

To study overfitting, models were trained with and without dropout.

Figure 5 compares training and validation accuracy for both cases.

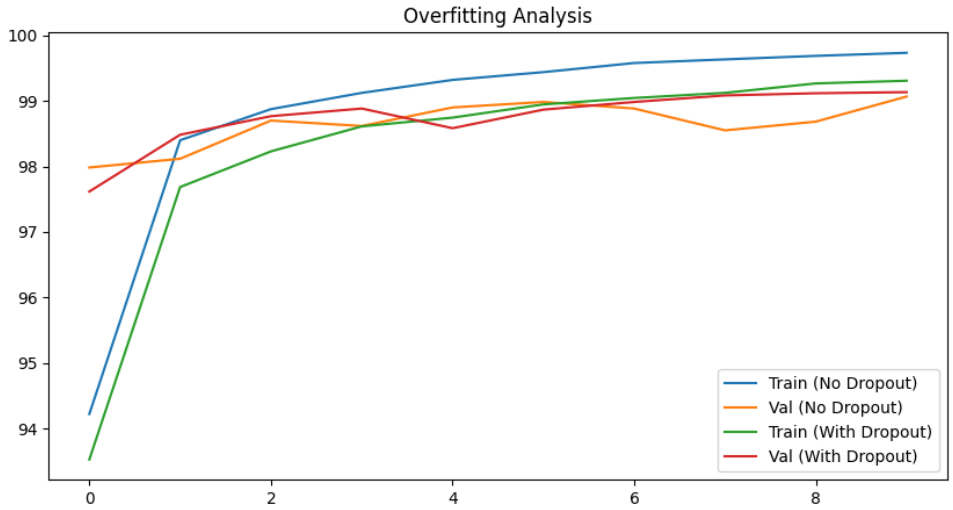


Figure 5: Effect of Dropout on Overfitting

It can be observed that dropout reduces overfitting. Although the training accuracy decreases slightly, the validation accuracy improves, indicating better generalization to unseen data. This demonstrates the regularization effect of dropout.

## 7 Hyperparameter Search and Best Model

A random search strategy was used to optimize batch size, learning rate, number of convolutional layers, and dropout usage.

Each trial was trained for 3 epochs to reduce computational cost. The best configuration was then retrained for 10 epochs and evaluated on the test set.

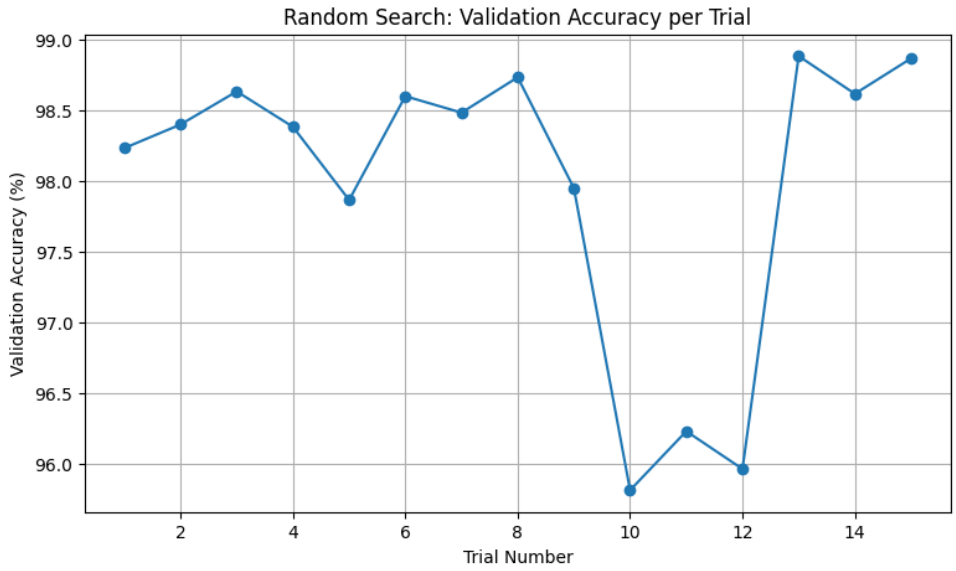


Figure 6: Best Hyperparameters Search

### 7.1 Best Hyperparameter Configuration

The random search identified the following configuration as the best-performing model:

- Batch Size: 64
- Learning Rate: 0.001
- Number of Convolutional Layers: 2
- Filters: [64, 128]
- Activation Function: ReLU
- Dropout: Enabled

The selected configuration achieved a validation accuracy of **98.88%**. After retraining the model using the full training set, the final test accuracy obtained was **99.23%**.

## 8 Pooling Layer Observations

### 8.1 Translation Invariance

The outputs were nearly identical, confirming the translation invariance property of the max pooling layer.

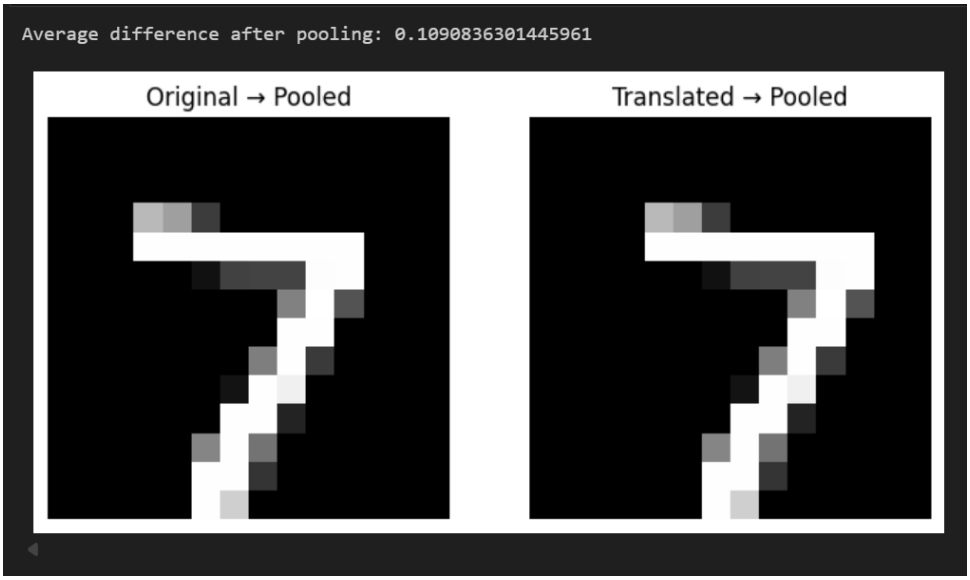


Figure 7: Observing the invariance of the pooling property

### 8.2 Backpropagation Through Max Pooling

A toy example was used to show that during backpropagation, gradients flow only through the maximum activation within each pooling window.

```
x_demo = torch.tensor(
    [[[[1., 2.],
       [3., 4.]]]],
    requires_grad=True,
    device=device
)
pool_back = nn.MaxPool2d(2)

y_demo = pool_back(x_demo)
print("Pooled output:", y_demo)
y_demo.backward(torch.ones_like(y_demo))

print("Gradient w.r.t input:\n", x_demo.grad)
```

```
Pooled output: tensor([[[[4.]]]], device='cuda:0', grad_fn=<MaxPool2DWithIndicesBackward0>)
Gradient w.r.t input:
 tensor([[[[0., 0.],
           [0., 1.]]]], device='cuda:0')
```

Figure 8: Backpropagation gradients

## 9 Conclusion

In this experiment, a Convolutional Neural Network was successfully implemented for handwritten digit classification on the MNIST dataset. The baseline CNN achieved high accuracy, demonstrating the effectiveness of convolutional architectures for image recognition tasks.

Through systematic experimentation, it was observed that increasing network depth slightly improves performance, while dropout acts as an effective regularization technique to reduce overfitting. Hyperparameter tuning further improved the model, achieving a final test accuracy of 99.23%.

Additionally, the translation invariance property and backpropagation behavior of the max pooling layer were successfully demonstrated. Overall, the experiments validate the robustness and effectiveness of CNNs for image classification problems.